Attention

Advanced Machine Learning

Sam Ogden

DRAFT

This slide deck is still under active development. Expect changes before it is finalized.

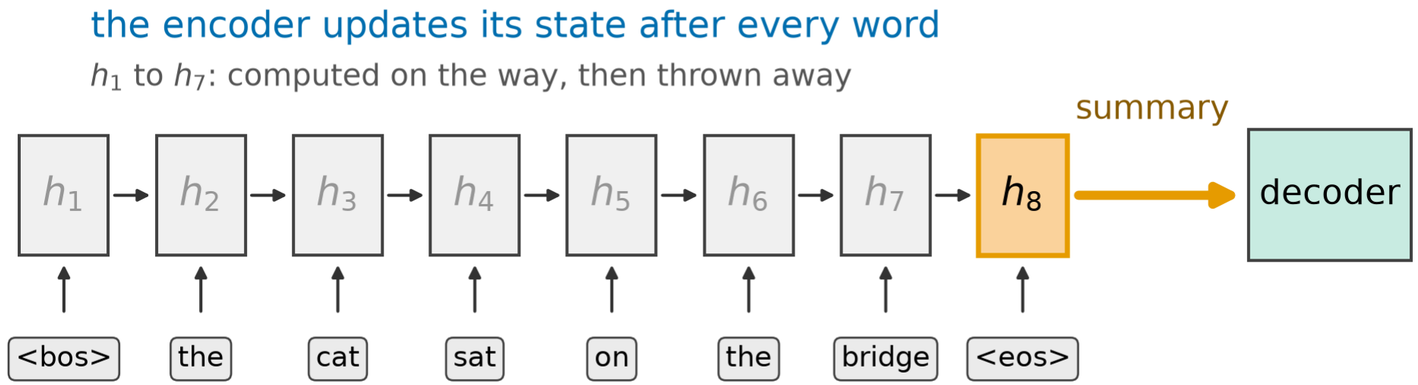
Learning objectives

- Explain why one fixed-size summary limits sequence to sequence, and what attention changes
- Name the query, the keys and the values in an attention step
- Compute attention scores, weights and a context vector by hand
- Read an attention heatmap
- Compare attention with a skip connection
- Name the main kinds of attention and when each is useful

seq2seq: one state only

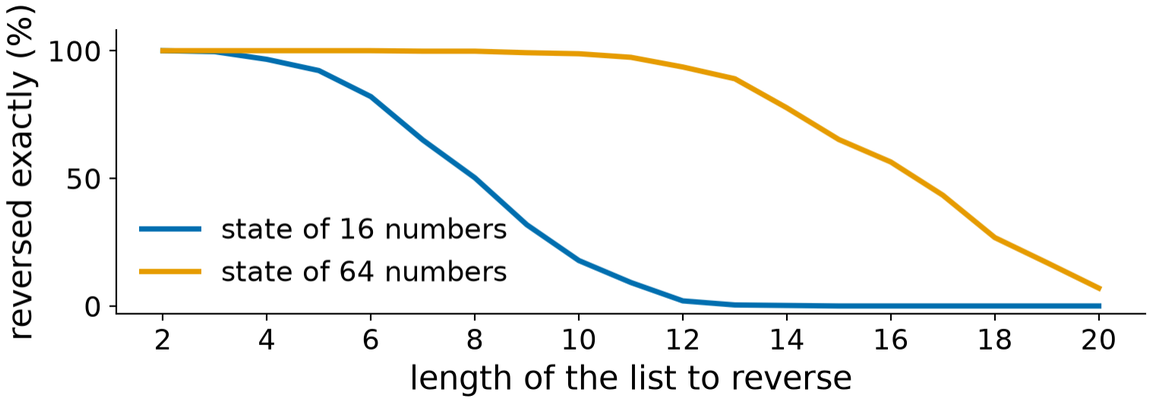
What the decoder gets, and what it never sees

How the decoder gets its input



- The encoder reads the English one word at a time. After each word it updates its state, so h_j is built from word j and the previous state h_{j-1}
- Only the last state, h_8, goes to the decoder: the **summary**. The earlier states were computed on the way, then thrown away

The problem: one state for everything



- Last time's test: read a list of digits, then write it backwards, with **only** the final state. Longer lists fail, and a bigger state only delays it

The question:

The encoder had a state at every word. Can the decoder use all of them, and the right one for the digit or word it writes?

The Big Idea

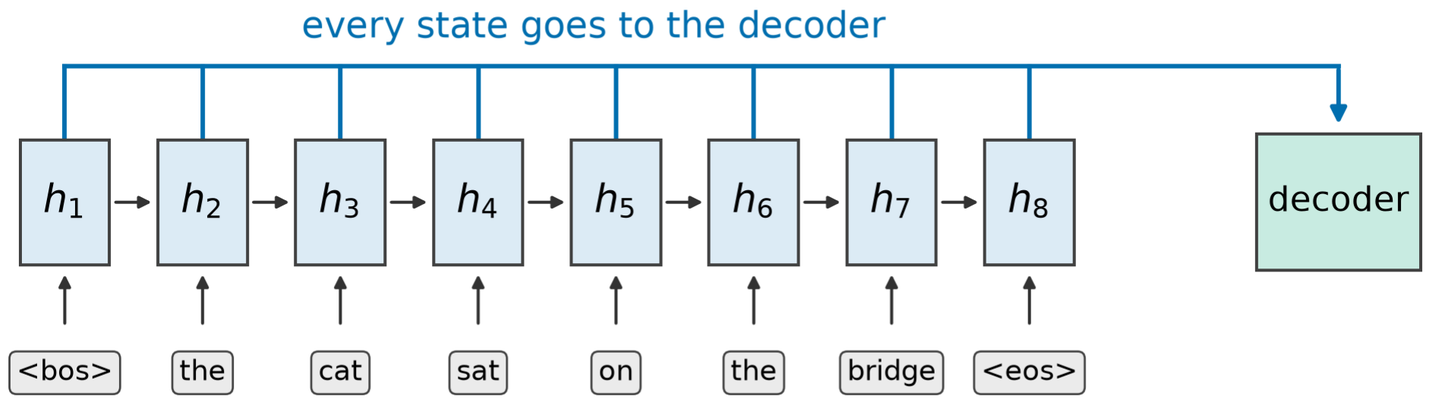
At every step, let the decoder score each encoder state by how relevant it is right now, then read a weighted average of them.

- The weights are worked out fresh for each word the decoder writes, from the content
- Skip connections handed the decoder a fixed copy. This look-back **chooses**

Attention, step by step

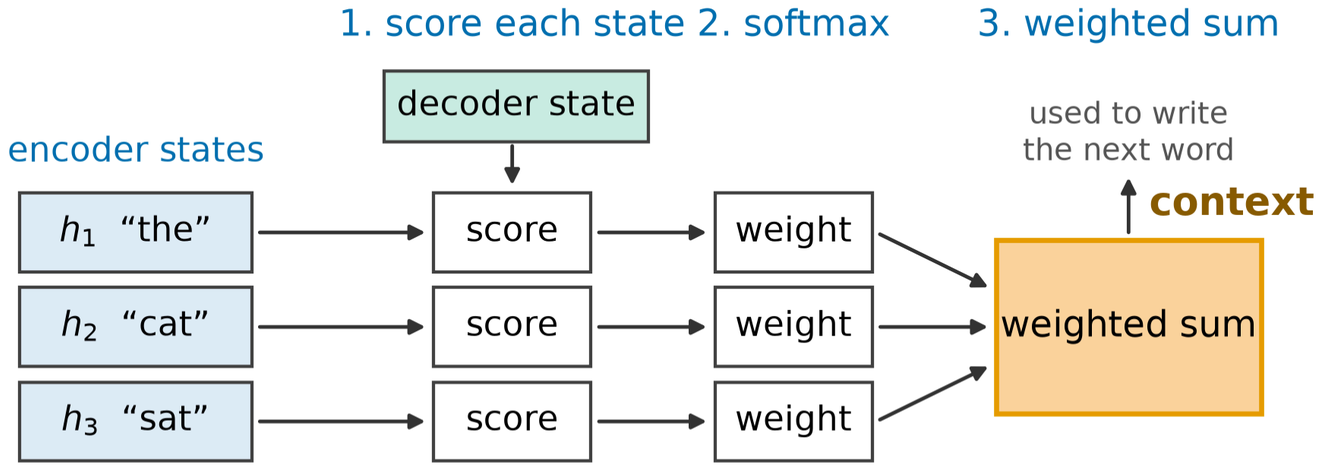
Scores, weights, and a weighted average

What the encoder hands over



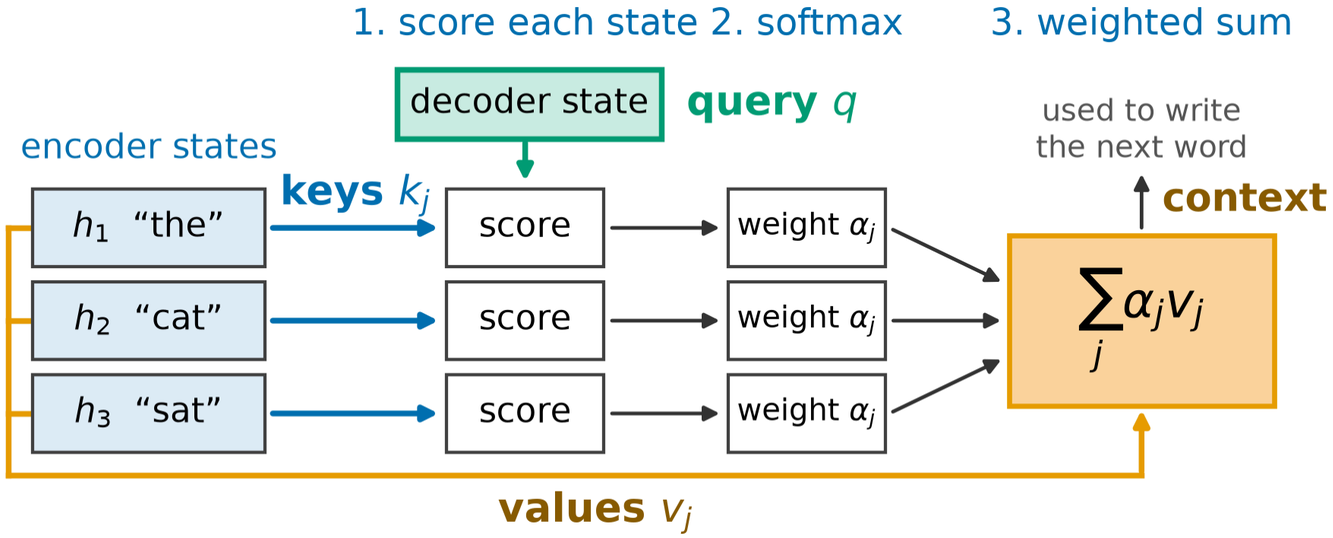
- Keep **every** state, one per word: h_1 to h_8. The decoder gets the whole set, not only the last
- The set grows with the input: eight words, eight states. Nothing has to be squeezed into one vector

One decoder step



- A small example: the encoder read "the cat sat" and produced three states
- Each encoder state is scored against the decoder's own state. The scores become weights, and the weighted sum of the states is the **context**

Queries, keys and values



- **Query** q: the decoder's state, asking "what do I need right now?"
- **Keys** k_j: the encoder states, matched against the query to give the weights
- **Values** v_j: what actually gets read. Here they are the same h_j as the keys, scaled by the weights and added up to give the context

The attention formula

$$\text{score}_j = \frac{q \cdot k_j}{\sqrt{d}} \qquad \alpha_j = \frac{e^{\text{score}_j}}{\sum_i e^{\text{score}_i}} \qquad c = \sum_j \alpha_j \, v_j$$

- **Score**: how well key j matches the query
- **Weight** α_j: the score as a share, from a softmax
- **Context** c: the values, averaged by weight
- d: the size of a state ($d = 64$ in our model)

- Three steps, taken in order on the next slides, with a tiny example: three words and made-up states of two numbers

Scores: how well a key matches

$$\text{score}_j = \frac{q \cdot k_j}{\sqrt{d}} \qquad q = [1, 3],\ d = 2$$

Word	Key k_j	Dot product $q \cdot k_j$	Score
the	[2, 0]	1×2 + 3×0 = 2	2 ÷ √2 = 1.41
cat	[0, 2]	1×0 + 3×2 = 6	6 ÷ √2 = 4.24
sat	[1, 1]	1×1 + 3×1 = 4	4 ÷ √2 = 2.83

- The dot product is large when two vectors point the same way. This query matches “cat” best
- Dividing by $\sqrt{d}$ keeps the scores from growing as the states get bigger

Weights: softmax

$$\alpha_j = \frac{e^{\text{score}_j}}{\sum_i e^{\text{score}_i}}$$

Word	Score	Exponential	Weight α_j
the	1.41	$e^{1.41} = 4.1$	4.1 ÷ 90.6 = 0.045
cat	4.24	$e^{4.24} = 69.6$	69.6 ÷ 90.6 = 0.768
sat	2.83	$e^{2.83} = 16.9$	16.9 ÷ 90.6 = 0.187

- The same softmax as in classification: every weight is positive and they add up to 1
- A bigger score takes a bigger share. “cat” gets 77% of the attention

Context: a weighted average

$$c = \sum_j \alpha_j \, v_j$$

$$= 0.045 \cdot [2, 0] + 0.768 \cdot [0, 2] + 0.187 \cdot [1, 1] = [0.277, 1.723]$$

- Each state is scaled by its weight and the results are added. Here “cat” dominates, so the context is close to the state of “cat”, $[0, 2]$
- The context has the **size of one state**, however long the source is. What changed is that it is picked anew at every step

A step with real numbers

One decoder step of our trained translation model: writing “chat”, with all eight encoder states of “the cat sat on the bridge” to look at:

	<bos>	the	cat	sat	on	the	bridge	<eos>
Score	-1.5	-1.0	3.5	-0.3	-1.7	-2.4	1.3	-0.6
Weight	0.01	0.01	0.85	0.02	0.00	0.00	0.09	0.01

- Eight encoder states, eight scores, and a softmax over them. The weights add up to 1 and **85%** is on “cat”, the word being translated
- The context is the weighted average of the eight 64-number states, which then helps write “chat”. Every word the model writes gets its own row of weights like this one

Attention in PyTorch

```
# the same model as before, plus attention (padding is masked out; left out here)
def forward(self, src, tgt_in):

    # encoder: a state for every source word
    states, state = self.encoder(self.src_embedding(src))

    # decoder: a state for every word it has written
    dec, _ = self.decoder(self.tgt_embedding(tgt_in), state)

    # 1. score: every decoder state against every encoder state
    scores = dec @ states.transpose(1, 2) / math.sqrt(states.shape[-1])

    # 2. softmax: over the source words, so each row adds up to 1
    weights = scores.softmax(dim=-1)

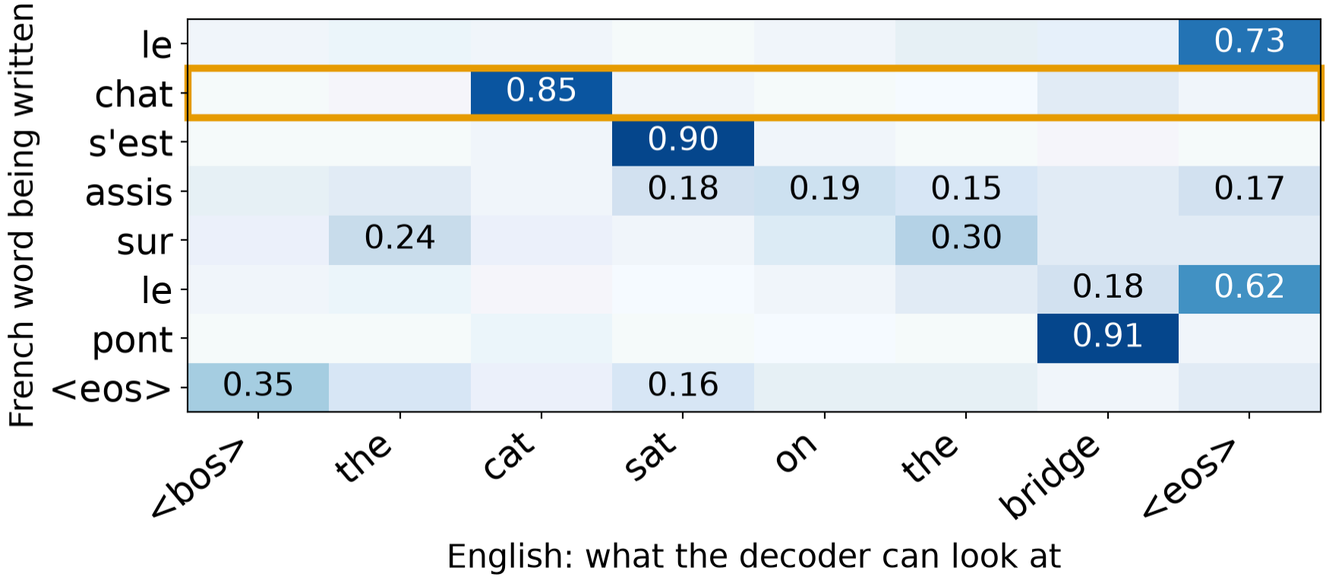
    # 3. weighted sum: the context is a weighted average of the encoder states
    context = weights @ states

    # the output layer sees the decoder state and the context side by side
    return self.out(torch.cat([dec, context], dim=-1))
```

What attention does

Where it looks, and what it fixes

The attention heatmap



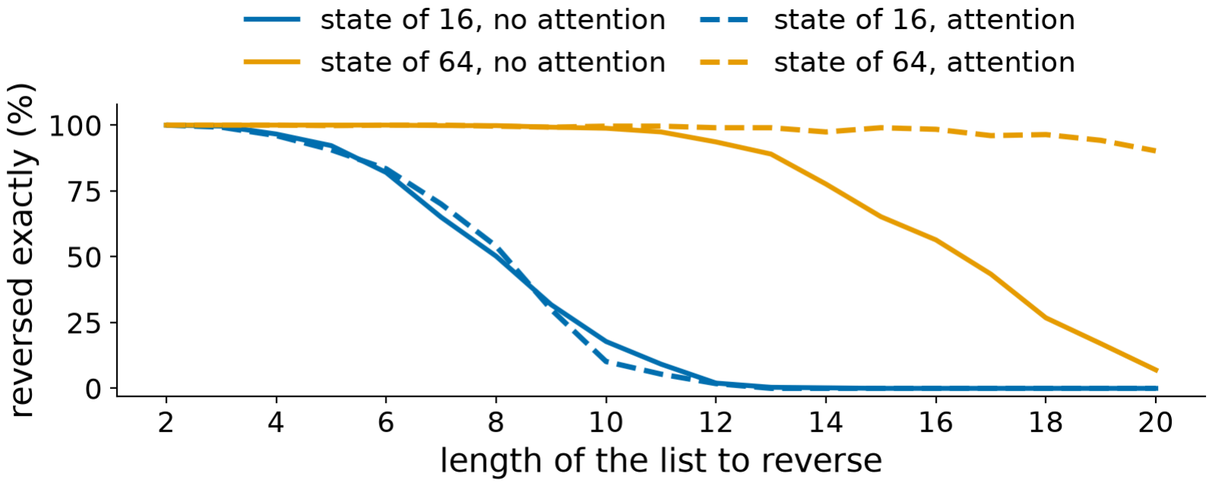
- Each **row** is one step, the French word being written. Read across it
- A row is one softmax, so it **adds up to 1**. Darker is more weight. The outlined row is the last slide's
- **Columns** add up to nothing in particular. The first "le" mostly uses <eos>

Translation with attention

Model	Unseen sentences right	Parameters
Plain: final state only	88% (80% to 95%)	37,224
With attention	100% (100% to 100%)	38,760

- The same 140 training pairs and the same 40 unseen sentences. First run, with the range over five runs in brackets
- The plain model’s mistakes were the place word. With attention, **no** unseen sentence was wrong in any of the five runs
- For the animal and the place word, the largest weight was on the matching English word in every unseen sentence, in all five runs

The bottleneck again



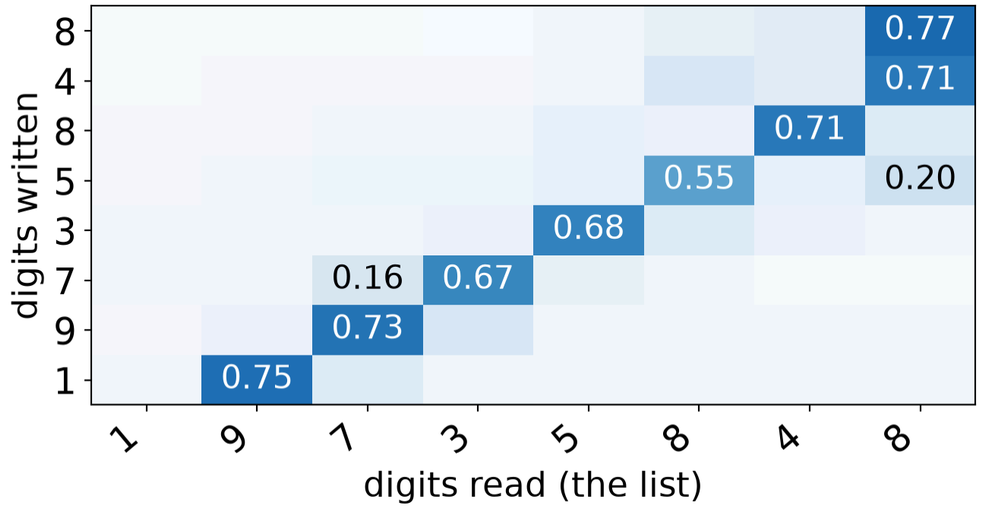
- The same test, now with attention (dashed). For 20-digit lists, a state of 64 numbers reverses **90%** of them with attention and 7% without
- With a state of 16 numbers, attention was no real help: both fail by length 10. The next slide varies the state size

How big do the states need to be?

Numbers in each state	12-digit lists reversed	20-digit lists reversed
16	2%	0%
32	36%	0%
48	97%	59%
64	99%	90%

- Attention helps more as each state gets bigger. With 48 numbers per state, 59% of 20-digit lists were reversed exactly, and with 64, 90%

Attention for reversing a list of digits



- The earlier list-reversing test, now with attention. It read **1 9 7 3 5 8 4 8** and wrote **8 4 8 5 3 7 9 1**
- Rows are digits written, columns are digits read: each step looks a place earlier
- Each peak is one column after the digit being written

Attention versus a skip connection

Earlier, in the image encoder-decoder, **skip connections** copied encoder feature maps across to the decoder.

	Skip connection	**Attention**
Reads from	The same-size encoder map, same position	Any encoder position, chosen by the query
What decides	The wiring, set in advance	The content, rescored at every step
The decoder gets	The whole map, stacked on its features	One weighted average

- Both give the decoder a way back to the encoder: fixed by position for a skip, chosen by content for attention

Takeaway

Attention scores every encoder state against what the decoder needs now, and reads a weighted average, so the decoder is no longer limited to one summary.

Kinds of attention

Same idea, different sources and scores

Where the queries and keys come from

Kind	Queries	Keys and values	Used for
Cross-attention (ours)	Decoder	Encoder	Translation, captions
Self-attention	A sequence	The same sequence	Reading in context
Masked self-attention	A sequence	Earlier positions of it	Generating text: no peeking ahead

- Self-attention and masking come next. They need no separate encoder and decoder

Other choices

Variation	What changes	When it helps
Dot-product (ours)	$q \cdot k / \sqrt{d}$ for every pair at once	Fast on GPUs. Needs same-size q and k
Additive	A small network scores each pair	Different-size q and k (D2L's Bahdanau)
Multiple heads	Several attentions in parallel	Each head can look for something different
Local or sparse	Score only a nearby window	Long inputs, where all pairs cost too much

Summary

Recap: what the decoder sees

	Plain	With attention
The decoder gets	The final state only	A weighted average of every encoder state
Weights	None	Softmax of scores, new at every step
Unseen sentences right	88%	100%
20-digit lists reversed	7%	90%

- The reversal rows use states of 64 numbers

Recap: key ideas

- The encoder has a state at every word. Attention lets the decoder read from **all** of them, not only the last
- A **query** (the decoder's state) is scored against the **keys** (the encoder states). A softmax turns the scores into weights, and the **values** are averaged by them
- The weights are recomputed at every step, so different output words look at different input words, as the heatmap shows
- On our toy it fixed every unseen sentence, and states of 64 numbers kept reversing long lists. States of 16 numbers did not benefit
- Attention comes in kinds: cross-attention (what we built), self-attention and masked self-attention, with different ways to score

Where this goes next

- **Self-attention** in depth: every word looks at every other word in the **same** sentence, with several heads and a way to tell positions apart
- With that, the recurrence is no longer needed, and we have the transformer
- The cost to watch: a score for every pair of positions, which grows quickly with length
- Not covered here: positional information, how masking is applied, and how heads are combined